

Arabic Text Summarization Using Seq2Seq with Attention

1. Introduction

This paper is about building a system that uses deep learning to summarize Arabic texts. Specifically, we worked on creating short headlines from full Arabic news articles, This kind of summary, called "abstractive," is extra hard for Arabic because the language has so many different word forms, is written right-to-left, and has many dialects. We built this system using something called a Sequence-to-Sequence (Seq2Seq) setup, which also includes a Bahdanau (or "additive") attention mechanism. We trained it using a big dataset that we put together from four public Arabic news sources. We did everything ourselves from scratch, like getting the data ready, breaking it into pieces (tokenization), building the model, and training it, all in Python using PyTorch.

1.1 Objectives

Our main goals were to gather several different Arabic natural language processing datasets and prepare them so they could all be used together for training. We also aimed to create our own Seq2Seq model, which has a special part called a Bidirectional LSTM encoder and another part, a Bahdanau attention decoder. We wanted to train the model carefully, stopping early if it started to "overfit" (meaning it memorized the training data too much), and we also wanted to save the version of the model that worked best. Finally, we planned to check how well the model performed using standard measures called ROUGE-1 and ROUGE-L.

2. Datasets

We put together our training data from four different sets of Arabic news summaries. Each of these sets included the complete news article along with its summary or headline. These datasets covered various dialects, topics, and sizes, which helped us create a varied dataset that the model could learn from and apply generally.

Dataset	Format	Split	Size	Content
SumArabic 1.0	JSONL	Pre-split	2,354 / 138 / 136	Arabic news (MSA) — columns: text, headline
AraSum	TSV	Re-split 85/15	Large corpus	Arabic news with abstractive summaries
Egyptian News	Parquet	Re-split 85/15	Moderate	Egyptian dialect — column: summarized_text
Arabic	CSV	Re-split	Moderate	Arabic summarization —

Kaggle		85/15		column: summarizer
--------	--	-------	--	--------------------

2.1 Dataset Merging Strategy

Since three of the datasets—AraSum, Egyptian News, and Arabic Kaggle—didn't come with their own training, testing, or validation sets, we first combined them all. Then, we split this big chunk: 85% for training and 15% for testing. After that, we took the training part and split it again, 85% for actual training and 15% for validation. The SumArabic dataset already had its own splits, so we simply added those to our newly created training, validation, and test sets. In the end, our training set had 46,911 pairs of articles and headlines. Both the validation and test sets each had roughly 8,000 or more pairs.

3. Data Preprocessing

3.1 Text Cleaning

Before we could use the text, we ran a special cleaning tool on both the original articles and the headlines we wanted to generate. This tool fixes common issues, including weird characters (Unicode noise) and problems specific to Arabic text.

Extra whitespace	Collapsed via <code>re.sub(r' +', ' ', x)</code> .
Quoted strings	It removed the outer quotation marks but kept what was inside.
Arabic guillemets	like » and «, were completely taken out.
Latin characters	Any Latin characters were removed, as our model was only meant to work with pure Arabic.
Parenthetical numbers	such as '(3/٤)' were also removed.
Isolated single Arabic letters	We filtered out single, isolated Arabic letters to cut down on unnecessary "noise" in the text.
Newlines	were removed from everything.
Unicode control chars	Various hidden Unicode control characters were stripped away.
Punctuation normalization	It also standardized punctuation, turning things like ellipses into a single period, changing Arabic percent symbols to the standard '%', taking out thousands of separators, and making different dash types consistent.

3.2 Tokenization

breaking the text into smaller pieces ,we used the AraBART tokenizer. This is a special tool, based on the BART model, that was already trained on a huge collection of Arabic text. It uses

something called a SentencePiece vocabulary, which is quite good at handling the complex word structures of Arabic.

Tokenizer	moussaKam/AraBART (HuggingFace Transformers)
Source max length	1,024 tokens (truncated + padded to max_length)
Target max length	256 tokens (truncated + padded to max_length)
Output fields	The output from this process included 'input_ids' (the unique numbers for each token), 'attention_mask' (telling the model which tokens to pay attention to), and 'labels' (the token IDs for the target summary).

3.3 Sequence Length Truncation for Training

To make sure everything fit into the GPU's memory during training, we shortened the sequences even more before feeding them into the system. The input for the encoder, which processes the original article, was capped at 256 tokens. The input for the decoder, which generates the summary, was limited to 128 tokens. We also used a technique called "teacher forcing." This means that when the decoder was learning, it always got the correct previous work from the actual summary, rather than its own prediction, to help it stay on track.

4. Model Architecture

Our model is a custom-built Sequence-to-Sequence (Seq2Seq) system, made from scratch using PyTorch. It's essentially made up of two main parts: a Bidirectional LSTM encoder and an LSTM decoder that's enhanced with Bahdanau attention.

4.1 Hyperparameters

Vocabulary size	determined by the AraBART tokenizer.
Embedding dimension	The embedding dimension was set to 256.
Hidden state size	The hidden state size was 512.
Batch size	We processed data in batches of 16.
Max training epochs	The model was trained for a maximum of 10 "epochs," which are full passes through the training data.
Optimizer	We used the Adam optimizer with a learning rate of 0.001.
Loss function	For the loss function, we used CrossEntropyLoss, making sure to ignore any padding tokens.
Gradient clipping	We applied gradient clipping with a max norm of 1.0 to keep

	the gradients from "exploding" during training.
Early stopping patience	If the model didn't show any improvement in its performance for 3 consecutive epochs, we stopped the training early.
Device	We ran the training on a CUDA GPU, but it could switch to a regular CPU if a GPU wasn't available.

4.2 Encoder

the encoder part of our model worked like this: First, an embedding layer converted the numerical IDs of our tokens into 256-dimensional vectors. Then, a Bidirectional LSTM layer took these vectors and processed them, outputting hidden states that were 1024-dimensional. Finally, the hidden and cell states from both the forward and backward passes of the LSTM were combined and then used to set up the initial state of the decoder.

4.3 Decoder with Bahdanau Attention

At each step when the decoder was generating a word, it calculated an "attention-weighted context vector" based on all the outputs from the encoder.

```
score(t) = V · tanh( W1·encoder_outputs + W2·decoder_hidden(t) )
attn_weights = softmax( score, masked where src_mask == 0 )
context = sum( attn_weights × encoder_outputs, dim=1 )
lstm_input = concat( decoder_embedding, context )
dec_output, decoder_hidden = decoder_lstm( lstm_input, decoder_hidden )
logit = fc( dec_output ) → VOCAB_SIZE distribution
```

The process involved a few steps, including calculating scores, applying a softmax function to get attention weights (making sure to ignore padding), and then combining these weights with the encoder outputs to form the context. This context was then joined with the decoder's word embedding and fed into the decoder's LSTM. The LSTM then produced an output, which was finally turned into a distribution over all possible words in the vocabulary. It was important that padding tokens were ignored during this attention calculation, so the model wouldn't try to "attend" to empty spots.

4.4 Trainable Modules

encoder_embedding	nn.Embedding(VOCAB_SIZE, 256)
encoder_lstm	nn.LSTM(256, 512, bidirectional=True)
decoder_embedding	nn.Embedding(VOCAB_SIZE, 256)
decoder_lstm	nn.LSTM(256 + 1024, 512) — input = embedding + context
W1 (attention)	nn.Linear(1024, 512)

W2 (attention)	<code>nn.Linear(512, 512)</code>
V (attention)	<code>nn.Linear(512, 1)</code>
fc (output projection)	<code>nn.Linear(512, VOCAB_SIZE)</code>

The different parts of our model that learned during training included an embedding layer for the encoder and the encoder's Bidirectional LSTM. There was also an embedding layer for the decoder and the decoder's LSTM, which took both the word embedding and the attention context as input. Additionally, three linear layers (W1, W2, and V) were used in the attention mechanism, and a final linear layer (fc) projected the decoder's output to the full vocabulary size.

5. Training Procedure

5.1 Training Loop

During training, we used a technique called "teacher forcing." This meant that at every step, the decoder was given the correct previous word from the actual summary, instead of having to rely on its own guess. This helped make the training more stable, especially in the beginning. The training process involved: First, the original article sequences went through the Bidirectional LSTM encoder. Then, we set up the decoder's starting hidden state by combining the final states from the encoder. For each step where the decoder was trying to generate a word, it calculated attention, created a context vector, and produced an output "logit" (a raw prediction). We then calculated the cross-entropy loss by comparing this prediction to the actual summary, making sure to ignore any padding tokens. After that, we used "backpropagation" to compute the gradients, and we applied gradient clipping with a max norm of 1.0 to keep them from exploding before letting the Adam optimizer adjust all the model's parameters. We also calculated the "validation loss" in the same way, but without making any adjustments to the model's parameters.

5.2 Early Stopping

To keep the model from overtraining, we stopped if its performance on the validation set didn't get better for three epochs in a row. Before saving the model, we made sure to load back the best-performing version of its weights.

Patience	was set to 3 epochs, meaning even a tiny improvement counted.
Min delta	0.0 (any improvement count).
Checkpoint tracking	We kept track of the best model by making a full copy of the internal settings for all eight parts of our system.
Output file	The final saved model was named 'seq2seq_model.pth'.

6. Evaluation

6.1 ROUGE Metrics

To see how good our model was, we used ROUGE scores. These are the usual way to measure how well summarization systems work.

ROUGE-1	looks at how many single words (unigrams) overlap between the summary our model made and the actual summary.
ROUGE-L	checks for the longest common sequence of words between our summary and the real one
Evaluation sample	We did this evaluation on 50 randomly chosen pairs from our validation set.
Scorer library	We use a scoring tool called 'rouge_score' from Google.

6.2 Evaluation Notes

Our system calculates ROUGE scores on actual decoded Arabic text, not on the numerical IDs of the tokens. For each validation sample, we first run the model end-to-end to generate a predicted summary token by token, then we convert those token IDs back into readable Arabic text using the tokenizer's 'decode' function. The same thing is done for the reference summary, with the padding tokens filtered out. That way, the ROUGE scores truly reflect the linguistic quality of the summaries rather than just how token IDs happen to line up.

7. End-to-End Pipeline Summary

#	Stage	Details
1	Data Ingestion	loaded our four different datasets (which came in JSONL, TSV, Parquet, and CSV formats) into Pandas DataFrames.
2	Standardization	We standardized them by renaming all the summary columns to 'target' and all the article columns to 'text'.
3	Train/Test Split	We split the data: 85% for training, 15% for testing, and a further 15% of the training data became our validation set.
4	Text Cleaning	We cleaned the text in both the 'text' and 'target' columns using our special cleaning function.
5	Tokenization	we used the AraBART tokenizer, setting a maximum length of 1,024 tokens for the source articles and 256 for the target

		summaries.
6	Tensor Prep	We prepared the data for our model, truncating sequences to 256 tokens for the encoder and 128 for the decoder, applying the teacher-forcing shift, and creating a TensorDataset.
7	Model Training	We then started training the model, running for a maximum of 10 epochs with the Adam optimizer (learning rate 0.001), gradient clipping (max norm 1.0), and stopping early if performance didn't improve for three epochs.
8	Best Checkpoint	Once training was done, we restored the best-performing model state and saved it as 'seq2seq_model.pth'.
9	Evaluation	Finally, we evaluated the model using ROUGE-1 and ROUGE-L scores on 50 samples from the validation set, computed on decoded Arabic text.

8. Challenges & Limitations

We ran into a few challenges and limitations with this project: We had to cut down longer articles: While some articles were up to 1,024 tokens, we had to shorten them to 256 tokens for training. This means we might have lost some important information from those longer pieces. Different Arabic dialects: Our datasets included a mix of Standard Arabic, Egyptian dialect, and other sources. This variety could sometimes make it hard for the model to handle different word forms. No "beam search": We only used a simple "greedy decoding" method during training, and we didn't add a more advanced "beam search" technique for when the model generates summaries. Starting from scratch: Even though we used the AraBART tokenizer, we didn't use its powerful pre-trained weights for the model itself. If we had fully "fine-tuned" a pre-trained model, the quality would likely have been much better.

9. Future Work & Recommendations

For the future, here are some things we'd recommend or plan to work on: A major step would be to fully "fine-tune" a pre-trained model like AraBART or AraBERT, using HuggingFace's Seq2SeqTrainer. This would tap into over 139 million pre-trained Arabic parameters and likely lead to much better results. We should add "beam search decoding" when the model generates summaries, using a beam width of 4 to 8, which usually makes for higher quality outputs. We could try to increase the maximum length for source articles to 512 or even 1024 tokens. We could achieve this by using techniques like gradient checkpointing or mixed-precision training (FP16). Adding a "copy/pointer mechanism" would be useful. This would let the decoder directly copy less common names or terms from the original article, rather than trying to generate them. We could also explore using dialect identification as an extra signal to help the model better handle inputs that mix Standard Arabic and Egyptian dialects.

10. Conclusion

To sum it up, this project successfully built a full system for summarizing Arabic text. We covered everything from bringing in raw data from different sources, getting it ready, tokenizing it, training our model with early stopping, all the way to evaluating it using ROUGE scores. Our custom Seq2Seq model, enhanced with Bahdanau attention, gives us a strong starting point for abstractive summarization in Arabic, and we can clearly see how it works. Using the AraBART tokenizer was key because it correctly broke down Arabic words into smaller pieces, which is important for the language. The Bidirectional LSTM encoder helped the model understand the text by looking at it from both directions, and the attention mechanism let the decoder focus on the most important parts of the original article as it generated each new word. Looking ahead, the biggest improvement we could make would be to replace our current custom-built Seq2Seq model with a fully "fine-tuned" AraBART model, using HuggingFace's Seq2SeqTrainer. This would take advantage of its vast pre-training and would almost certainly lead to much better ROUGE scores on our test data.